

Neural networks for language modeling

- Motivations

Bi-gram $P(w_t|w_{t-1}) = \frac{\text{Count } [w_t, w_{t-1}]}{\text{Count } w_{t-1}}$

n -gram $P(w_t|[w_{t-1}, \dots, w_{t-n+1}]) = \frac{\text{Count } [w_t, \dots, w_{t-n+1}]}{\text{Count } [w_{t-1}, \dots, w_{t-n+1}]}$

The students walked in the room and asked the ___?___ about the quiz questions.

- The n -gram model needs to memorize these probabilities.
 - It has to see the sequences "*and asked the professor*" and "*and asked the*" sufficiently frequently to estimate $P(\text{"professor"}|\text{"and asked the"})$ well.
 - If no such sequence has been seen, the probability is 0, which is incorrect.
 - If such sequence are seen but not frequent enough, the estimated probability is inaccurate (law of large numbers).

Neural networks for language modeling

Three issues:

- Data sparsity: the occurrences of many token sequences $[w_t, \dots, w_{t-n+1}]$ are zeros.
- Model complexity: the number of parameters increases exponentially in n .
- Architecture: using a fix window of previous words is too rigid and straightforward.

The three issues are related: if we want longer range dependencies for more contexts, we increase n , then both the data sparsity and model complexity become worse. The simple ratios statistics won't be flexible to account for various ranges of dependencies.

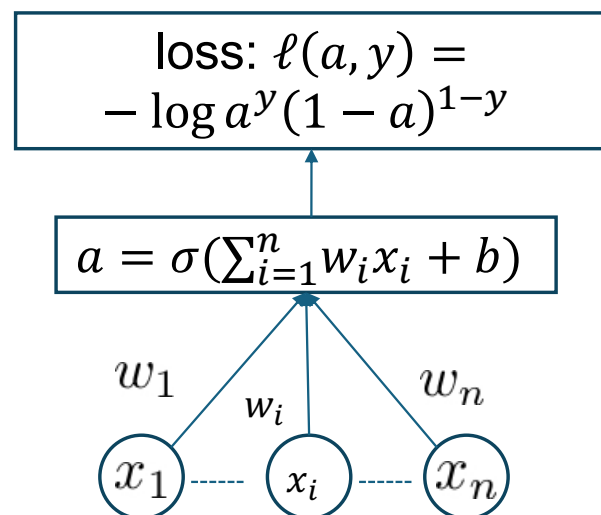
Neural networks as a solution

- Neural networks use a fixed set of parameters to make prediction, with the architecture to handle various length of dependencies.
 - Model parameters are used more efficiently.
 - Don't just increase the window size by 1 just to deal with longer dependencies.
 - Increase model size by a large factor while only to address some rarely seen word sequence.
 - Using more design elements to tackle linguistic variations, such as architecture, loss function, training tasks.
 - Recurrent neural networks: impose the sequential architecture on neural networks.
 - Transformers: attention architecture is all you need, and let the data tune the model to decide how to pay attention to various parts of a context.
 - SFT/RLHF: let the training tasks and loss function shape the neural networks.

Neural network review

- Logistic regression for classification (learned from DSAA 2011) is a neural network.

A computation graph is a differentiable system for **Forward** and **Back-propagation**.



Forward pass:

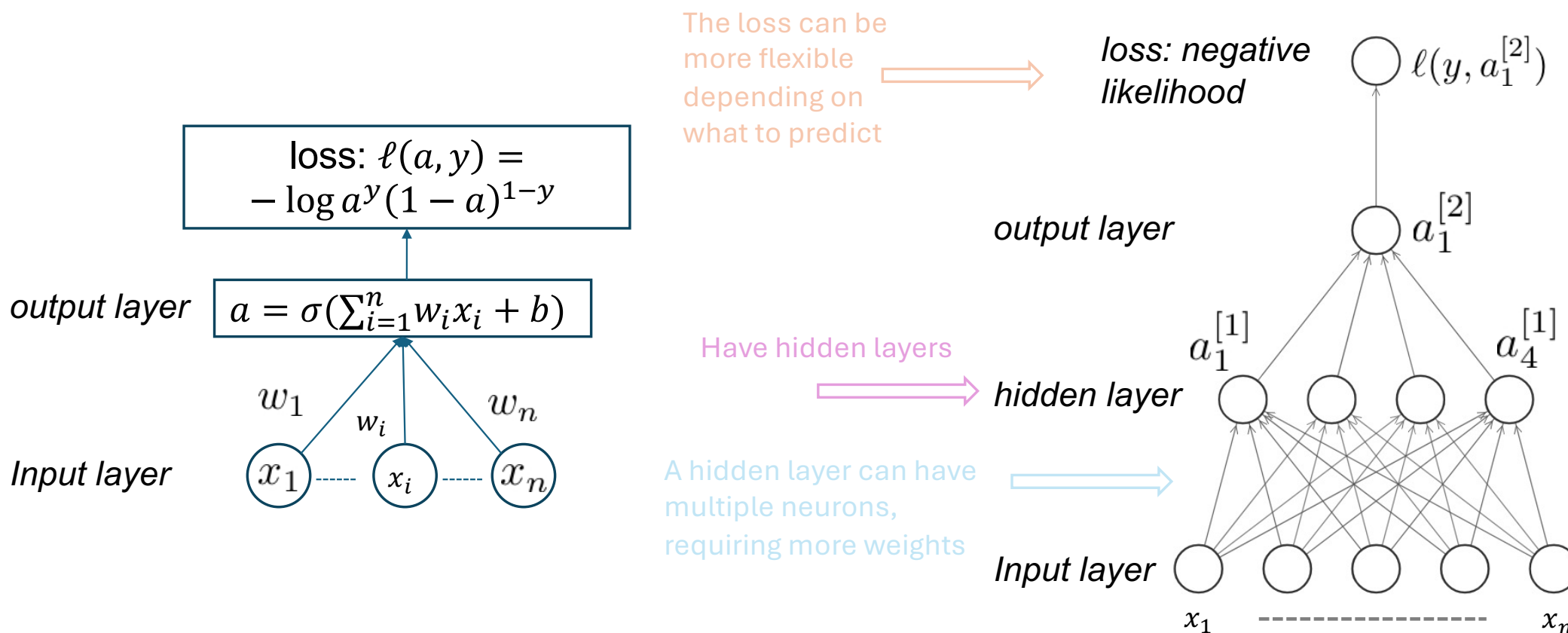
- Given input, compute the hidden and output units, and the loss.
- For logistic regression, the hidden state and the output are the same (the scalar a)

Back-propagation:

- compute the gradients of the loss with respect to parameters using the chain rules.

Neural network review

- From logistic regression to multi-layer perception



Neural network review

- A neural network is a computation graph that stacks many Logistic regression models.

This is a logistic regression model

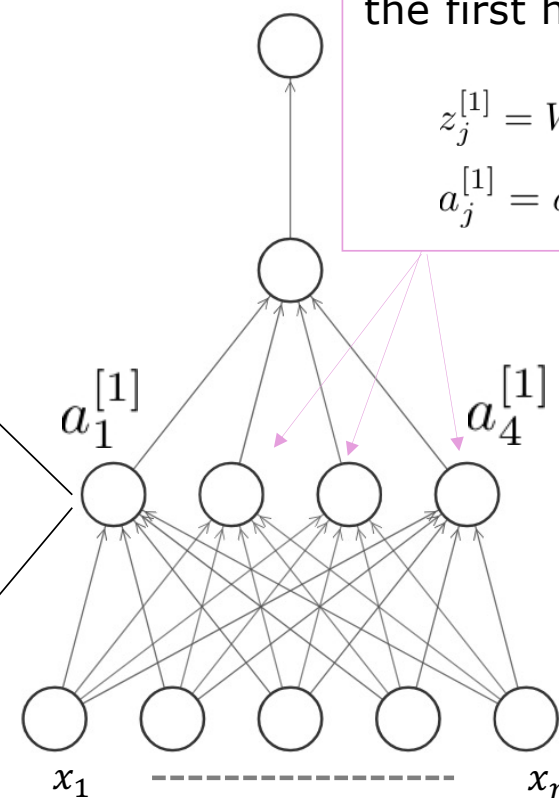
- Computer the inner product or logit

$$z_1^{[1]} = W_1^{[1]\top} x + b_1^{[1]}$$

$W_1^{[1]}$ the vector of weights on the first layer connecting x to $z_1^{[1]}$

- Computer non-linear mapping (sigmoid or ReLU) of the logit

$$a_1^{[1]} = \sigma(z_1^{[1]})$$



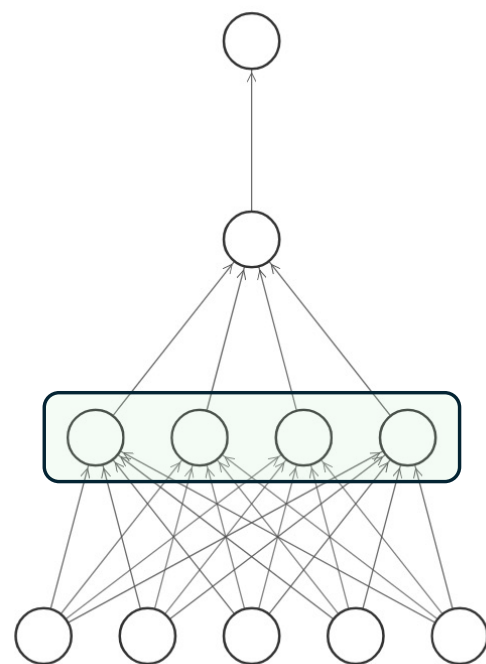
For the j-th neural on the first hidden layer:

$$z_j^{[1]} = W_j^{[1]\top} x + b_j^{[1]}$$

$$a_j^{[1]} = \sigma(z_j^{[1]})$$

Neural network review

- Vectorization: computing the hidden neurons one by one can be slow.



At the first hidden layer:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]})$$

4 units

$$\begin{bmatrix} W_1^{[1]\top} \\ W_2^{[1]\top} \\ W_3^{[1]\top} \\ W_4^{[1]\top} \end{bmatrix} \mathbf{x} + \mathbf{b}^{[1]}$$

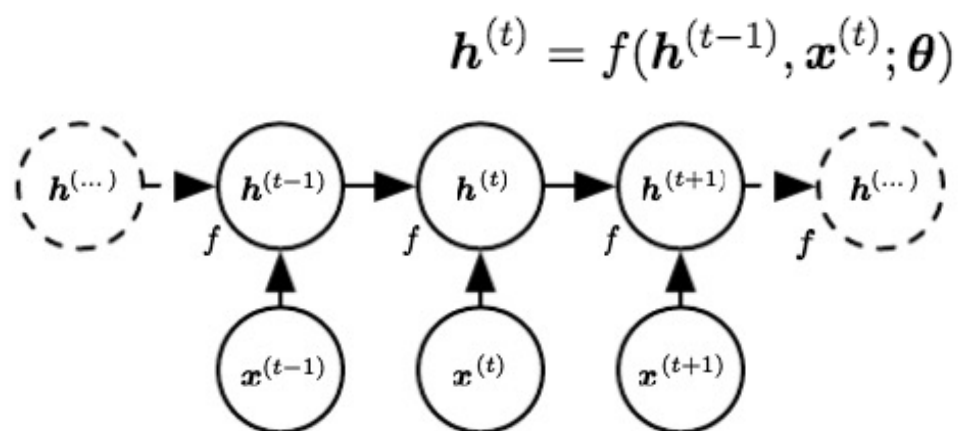
5 units

$$\mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]})$$

Recurrent neural networks

- Instead of memorizing probabilities, predict the probabilities using a model.
 - reduced data sparsity issue (probability is *not* memorized).
 - can generalize to unseen sequences (compression leads to intelligence).

Recurrent Neural Networks (RNN)



Fun facts: Before the "Transformer" architecture (the "T" in ChatGPT) took over the world in 2017, RNNs and LSTMs were the kings of the hill. Every time you used Google Translate or talked to a voice assistant between 2014 and 2018, you were interacting with a massive ensemble of Recurrent Neural Networks.

"Recurrent" because the next h is compute by the same function that calculates the previous h .

$h^{(t)}$ state summarizing what has happened before time step t . (theoretically speaking)

θ a single model specifying how to transit to the next state given the previous state and current input. (independent of t).

Recurrent neural networks

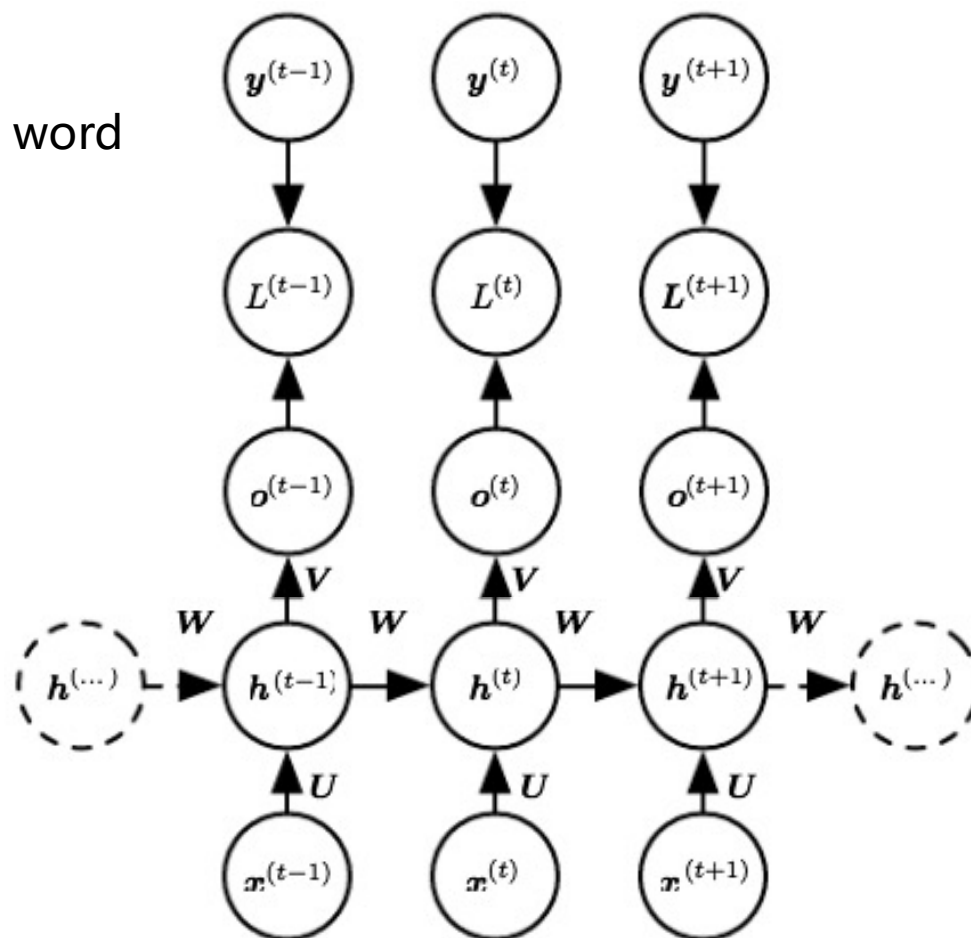
Output sequence
(e.g., predicted next word
or POS tag)

Loss functions

Output units
(probabilities of y^t)

Hidden states

Input sequence
(turned to their
embeddings)



Training data:

$$\{x^{(1)}, \dots, x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}$$

For language modeling: $y^t = x^{t+1}$

For POS tagging: $y^t = \text{POS}(x^{t+1})$

Trainable parameters:

$$\theta = \{U, W, V, b, c\}$$

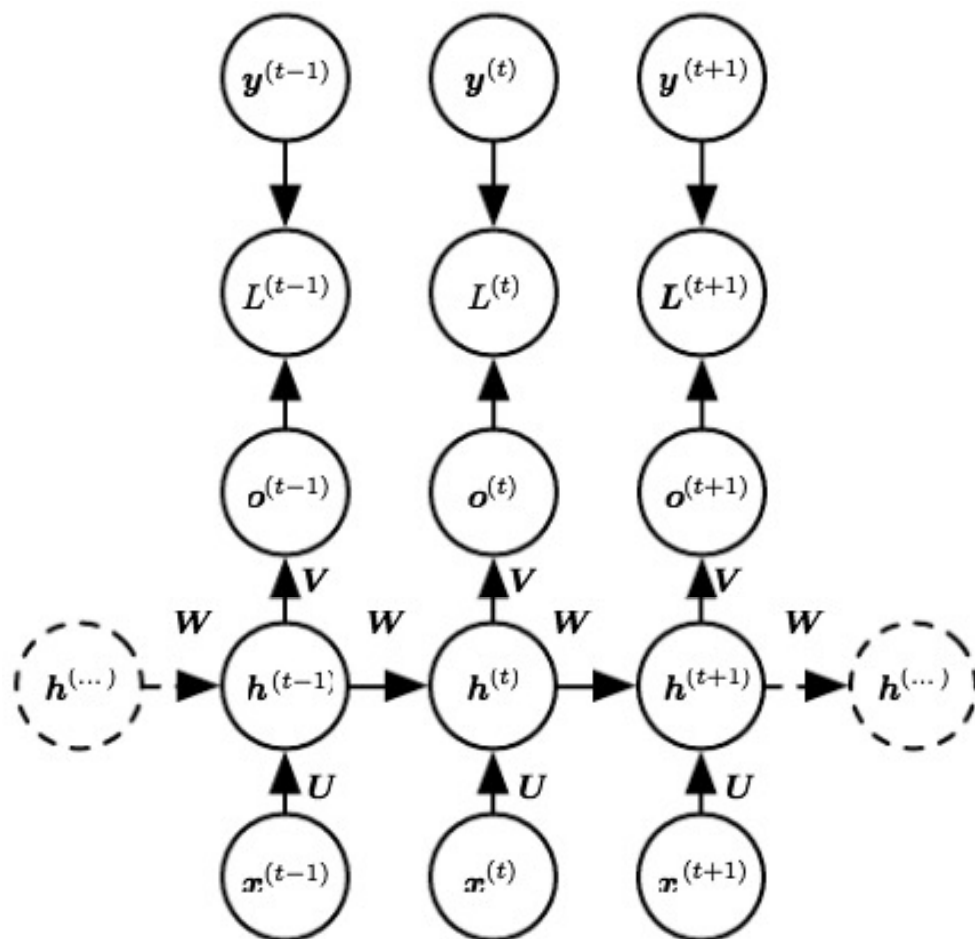
V : matrix maps h to o

W : matrix maps $h^{(t)}$ to an
element of $h^{(t+1)}$

U : matrix maps $x^{(t)}$ to an
element of $h^{(t+1)}$

b, c : biases

Recurrent neural networks



$$\mathbf{a}^{(t)} = \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)},$$

$$\mathbf{h}^{(t)} = \tanh(\mathbf{a}^{(t)}), \quad \text{hyperbolic activation}$$

$$\mathbf{o}^{(t)} = \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)},$$

$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)}), \quad \text{multi-class probability distribution}$$

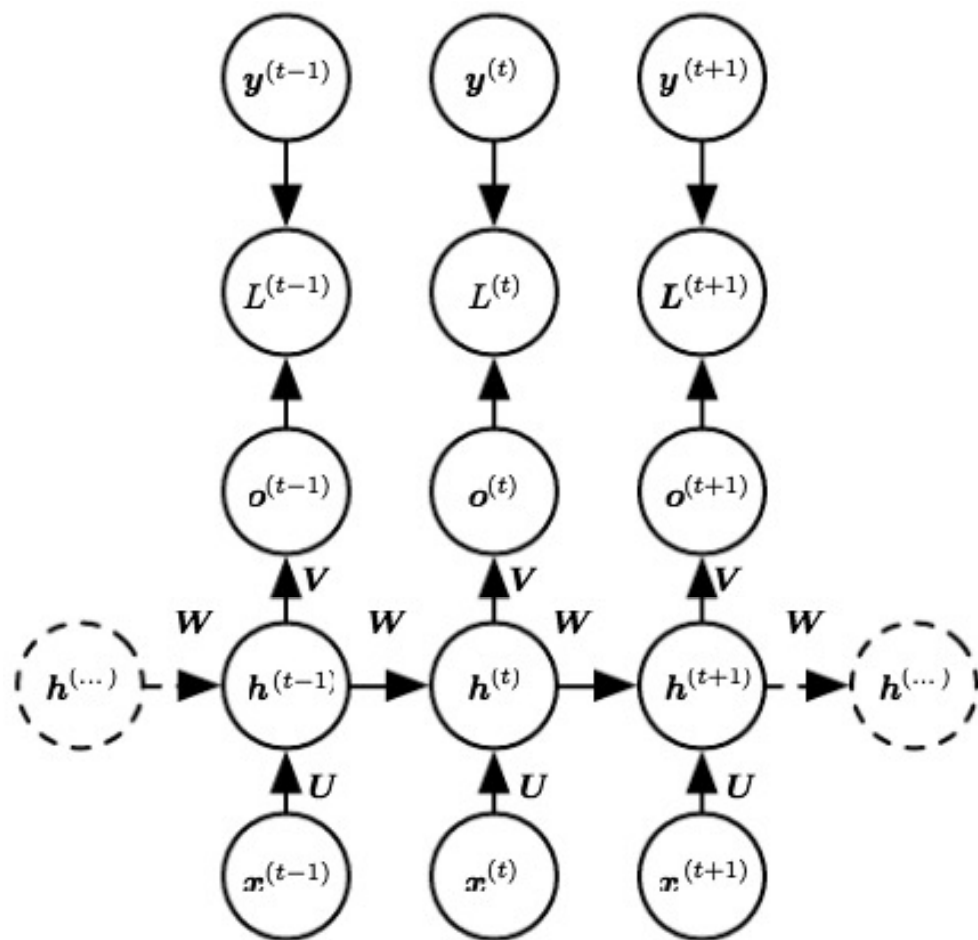
$$\mathbf{h}^{(t)} = \tanh(\mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)})$$

$$= \tanh(\mathbf{b} + \underbrace{\mathbf{W}(\tanh(\mathbf{b} + \mathbf{W}\mathbf{h}^{(t-2)} + \mathbf{U}\mathbf{x}^{(t-1)}))}_{\text{Information from token } x^{(t-1)} \text{ and all before time } t-1 \text{ summarized in the vector } h^{(t-2)}} + \mathbf{U}\mathbf{x}^{(t)})$$

Information from token $x^{(t-1)}$
and all before time $t - 1$
summarized in the vector $h^{(t-2)}$

Assimilate
information
from token x^t

Recurrent neural networks: training



Negative log likelihood (NLL) loss, or the “perplexity”

$$\begin{aligned}
 & L\left(\{x^{(1)}, \dots, x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}\right) \\
 &= \sum_t L^{(t)} \\
 &= - \sum_t \log p_{\text{model}}\left(y^{(t)} \mid \{x^{(1)}, \dots, x^{(t)}\}\right)
 \end{aligned}$$

the index of the ground truth POS tag.

$\hat{y}^{(t)} = \text{softmax}(o^{(t)})$

Back propagation is used to update the model parameters. The algorithm is called back-prop through time (BPTT). Challenge is about gradient vanishing or explosion (see extra reading to learn about this).

Why nonlinear mapping tanh?

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- A non-linear activation function is necessary for RNN
 - without that, the recursion just collapse into a linear mapping.

$$\begin{aligned} h^{(t)} &= b + W(b + Wh^{(t-2)} + Ux^{(t-1)}) + Ux^{(t)} = b + Wb + W^2h^{(t-2)} + WUx^{(t-1)} + Ux^{(t)} \\ &= b + W^2b + \dots + W^tb + W^tUx^{(0)} + \dots + WUx^{(t-1)} + Ux^{(t)} \end{aligned}$$

- Intuitively, languages cannot be modeled by just a linear function.
- Why tanh: the matrix exponential W^t is particularly problematic:
 - If the matrix has "small-ish" eigenvalues, the exponential converges to the zero matrix, erasing past information in the sentence.
 - The back-propagation algorithm is very sensitive to the eigenvalues, leading to gradient vanishing or explosion.
 - The tanh function squash values between -1 and 1, making memorization and optimization easier.

| Extra readings

- For neural network and RNN, see Chap 6 and 10 of Deep Learning by Ian Goodfellow.
- In particular, the long-range dependencies issue, the optimization challenge of RNN, and variations of RNN such as LSTM.

| Demon

- Define a RNN.
- Optimize RNN on a toy example corpus.

Conclusion

- Neural networks are compressing text data using a model of fixed size.
- More flexible than n-gram for language modeling but also more powerful for other NLP tasks.
- RNN is a sequential neural models that can model dependencies of words in a sentence.
- The choice of tanh activation function is strategic to make RNN work.

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T/F): n-gram language models cannot model long-range dependencies (in particular words that are n tokens away).
 - Q2 (T/F): RNN can be trained for language modeling only.
 - Q3 (short answer): If the input word embedding vector to an RNN has dimension m, and the hidden state has dimension n, what are the sizes of the matrix W?
 - A: T, F, $n \times n$